

USER STORY L4

Web & Database Server

Technical Documentation
Apache · MariaDB · PHP · phpMyAdmin

Student	SOFRA Jonathan
School	LPEM
Class / Year	PIF 25/26
Calendar Week	CW 10

1. Introduction

This document describes the installation and configuration of a combined web and database server on Ubuntu Server. The server hosts an Apache web server, a MariaDB database, PHP for server-side scripting, and phpMyAdmin as a graphical database management interface.

Objective: Provide a fully functional LAMP stack (Linux, Apache, MariaDB, PHP) accessible from the local network, with a test database, test website, and secure configuration.

1.1 Architecture Overview

The following components are installed on a single server:

- Ubuntu Server (pre-installed, configuration not documented here)
- Apache 2 — HTTP web server
- MariaDB — relational database management system
- PHP 8.x — server-side scripting language
- phpMyAdmin — web-based database management tool

2. System Update

Before installing any packages, it is essential to update the system's package list and upgrade all installed packages to their latest versions.

2.1 Update Package List and Upgrade

```
sudo apt update
sudo apt upgrade -y
```

Explanation of commands:

- `apt update` — Fetches the latest package metadata from the configured repositories. This does not install or upgrade anything.
- `apt upgrade -y` — Upgrades all installed packages to their newest available versions. The `-y` flag automatically confirms all prompts.

3. Apache Web Server Installation

Apache HTTP Server is one of the most widely used web servers in the world. It will serve the test website and host the phpMyAdmin interface.

3.1 Installation

```
sudo apt install apache2 -y
```

3.2 Enable and Start Apache

```
sudo systemctl enable apache2
sudo systemctl start apache2
```

Explanation of commands:

- `systemctl enable apache2` — Configures Apache to start automatically at system boot.
- `systemctl start apache2` — Starts the Apache service immediately without rebooting.

3.3 Verify Installation

To check that Apache is running correctly:

```
sudo systemctl status apache2
```

Test: Open a browser and navigate to `http://<server-ip>/`. The default Apache2 Ubuntu welcome page should be displayed, confirming that the server is reachable on the network.

3.4 Configure File Permissions

To prevent unauthorized users from accessing or modifying website files, the ownership and permissions must be configured correctly. The web root directory is `/var/www/html/`.

```
sudo chown -R www-data:www-data /var/www/html/  
sudo chmod -R 755 /var/www/html/
```

Explanation:

- `chown -R www-data:www-data` — Sets Apache's system user (`www-data`) as the owner of all files under the web root. This ensures that only Apache and administrators have write access.
- `chmod -R 755` — Grants the owner full read/write/execute permissions, while the group and others only have read and execute permissions. This prevents external write access to the website files.

4. MariaDB Database Server Installation

MariaDB is an open-source relational database management system, fully compatible with MySQL. It will be used to store the website's data.

4.1 Installation

```
sudo apt install mariadb-server -y
```

4.2 Enable and Start MariaDB

```
sudo systemctl enable mariadb  
sudo systemctl start mariadb
```

4.3 Secure the Installation — `mysql_secure_installation`

The `mysql_secure_installation` script guides the administrator through a series of security improvements for the database server. It is strongly recommended to run this after installation.

```
sudo mysql_secure_installation
```

The script will prompt for the following actions:

- Set or change the root password — Set a strong password for the MariaDB root account.
- Remove anonymous users — Removes default anonymous accounts that can be a security risk.
- Disallow root login remotely — Restricts root access to localhost only, preventing remote root access.
- Remove test database — Deletes the default 'test' database that is accessible to anonymous users.

- Reload privilege tables — Applies all changes immediately.

Answer 'Y' to all prompts during `mysql_secure_installation` to apply all recommended security hardening measures.

4.4 Create Test Database and Dedicated User

A test database is created along with a dedicated database user that has minimal, necessary privileges only (principle of least privilege).

```
sudo mariadb -u root -p

-- Create the test database
CREATE DATABASE testdb;

-- Create a dedicated user with a strong password
CREATE USER 'webuser'@'localhost' IDENTIFIED BY 'SecurePassword123!';

-- Grant only the necessary privileges on the test database
GRANT SELECT, INSERT, UPDATE, DELETE ON testdb.* TO 'webuser'@'localhost';

-- Apply the privilege changes
FLUSH PRIVILEGES;

-- Exit MariaDB
EXIT;
```

Explanation:

- `CREATE DATABASE testdb` — Creates the test database named 'testdb'.
- `CREATE USER 'webuser'@'localhost'` — Creates a new MariaDB user that can only connect from localhost.
- `GRANT SELECT, INSERT, UPDATE, DELETE` — Grants only the four standard data operations. The user cannot drop tables, create new tables, or access other databases.
- `FLUSH PRIVILEGES` — Forces MariaDB to re-read the grant tables and apply the new privileges immediately.

5. PHP Installation

PHP (PHP: Hypertext Preprocessor) is a widely-used server-side scripting language that allows dynamic web page generation. It is required for phpMyAdmin and for any dynamic website functionality.

5.1 Installation

```
sudo apt install php libapache2-mod-php php-mysql -y
```

Packages installed:

- php — The core PHP interpreter.
- libapache2-mod-php — The Apache module that allows Apache to process PHP files.
- php-mysql — The PHP extension that enables PHP to connect to a MariaDB/MySQL database.

5.2 Verify PHP

To confirm PHP is installed and working with Apache, create a test PHP file:

```
sudo nano /var/www/html/info.php
```

Add the following content:

```
<?php
phpinfo();
?>
```

Test: Navigate to `http://<server-ip>/info.php` in a browser. The PHP information page will display, confirming PHP is functioning. Remove this file after testing for security reasons: `sudo rm /var/www/html/info.php`

6. phpMyAdmin Manual Installation

phpMyAdmin is a free, open-source web interface for managing MariaDB/MySQL databases. It is installed manually from the official website to ensure the latest version is used.

6.1 Install Prerequisites

```
sudo apt install php-mbstring php-zip php-gd php-json php-curl -y
sudo phpenmod mbstring
sudo systemctl restart apache2
```

6.2 Download the Latest phpMyAdmin

Visit <https://www.phpmyadmin.net/downloads/> to find the latest version number. At the time of this documentation, replace the version number below with the most current release.

```
cd /tmp
wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz
```

6.3 Extract and Move to Web Root

```
tar -xzf phpMyAdmin-latest-all-languages.tar.gz
sudo mv phpMyAdmin-*all-languages /usr/share/phpmyadmin
```

6.4 Create Apache Configuration for phpMyAdmin

```
sudo nano /etc/apache2/conf-available/phpmyadmin.conf
```

Add the following content to the configuration file:

```
Alias /phpmyadmin /usr/share/phpmyadmin

<Directory /usr/share/phpmyadmin>
    Options SymLinksIfOwnerMatch
    DirectoryIndex index.php
    AllowOverride All
    Require all granted
</Directory>
```

6.5 Enable Configuration and Restart Apache

```
sudo a2enconf phpmyadmin
sudo systemctl restart apache2
```

6.6 Configure phpMyAdmin

Copy the sample configuration file and set a blowfish secret for session encryption:

```
sudo cp /usr/share/phpmyadmin/config.sample.inc.php
/usr/share/phpmyadmin/config.inc.php
sudo nano /usr/share/phpmyadmin/config.inc.php
```

Find the line containing `$cfg['blowfish_secret']` and add a random 32-character string:

```
$cfg['blowfish_secret'] = 'YourRandom32CharacterStringHere!';
```

6.7 Create Temporary Directory and Set Permissions

```
sudo mkdir /usr/share/phpmyadmin/tmp
sudo chown -R www-data:www-data /usr/share/phpmyadmin/
sudo chmod 700 /usr/share/phpmyadmin/tmp
```

Test: Navigate to `http://<server-ip>/phpmyadmin` in a browser. Log in using the MariaDB root account or the 'webuser' account created earlier. phpMyAdmin should load successfully.

7. Test Website Creation

A simple test website is created to verify that Apache is correctly serving PHP pages and can connect to the MariaDB database.

7.1 Create the Website Files

```
sudo nano /var/www/html/index.php
```

Add the following PHP code to test the database connection:

```
<?php
$servername = 'localhost';
$username   = 'webuser';
$password   = 'SecurePassword123!';
$dbname     = 'testdb';

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die('Connection failed: ' . $conn->connect_error);
}

echo '<h1>LPEM - SOFRA Jonathan</h1>';
echo '<p>Web and database server is running!</p>';
echo '<p>Connected to database: ' . $dbname . '</p>';

$conn->close();
?>
```

7.2 Set File Ownership

```
sudo chown www-data:www-data /var/www/html/index.php
sudo chmod 644 /var/www/html/index.php
```

Test: Browse to <http://<server-ip>/> — the page should display the student name, confirmation that the server is running, and that the database connection was successful.

8. HTTPS with Self-Signed Certificate (Optional)

To encrypt traffic between the browser and the server, a self-signed SSL certificate can be issued and the virtual host configured to use HTTPS. HTTP requests are redirected to HTTPS automatically.

8.1 Enable SSL Module

```
sudo a2enmod ssl
sudo a2enmod rewrite
sudo systemctl restart apache2
```

8.2 Generate the Self-Signed Certificate

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
```

```
-keyout /etc/ssl/private/apache-selfsigned.key \  
-out /etc/ssl/certs/apache-selfsigned.crt
```

Fill in the prompts with appropriate values (Country, State, Organization, etc.). The Common Name should match the server's IP address or hostname.

8.3 Configure the HTTPS Virtual Host

```
sudo nano /etc/apache2/sites-available/default-ssl.conf
```

Configure the virtual host with the certificate paths:

```
<VirtualHost *:443>  
    ServerAdmin webmaster@localhost  
    DocumentRoot /var/www/html  
    SSLEngine on  
    SSLCertificateFile      /etc/ssl/certs/apache-selfsigned.crt  
    SSLCertificateKeyFile   /etc/ssl/private/apache-selfsigned.key  
</VirtualHost>
```

8.4 Redirect HTTP to HTTPS

Edit the default HTTP virtual host to redirect all traffic to HTTPS:

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

Add the redirect rule inside the `<VirtualHost *:80>` block:

```
<VirtualHost *:80>  
    ServerName <server-ip>  
    Redirect permanent / https://<server-ip>/  
</VirtualHost>
```

8.5 Enable SSL Site and Restart

```
sudo a2ensite default-ssl  
sudo systemctl restart apache2
```

Result: Navigating to `http://<server-ip>/` will automatically redirect to `https://<server-ip>/`. The browser will show a security warning because the certificate is self-signed — this is expected and can be bypassed by accepting the risk in the browser.

9. Installation Summary

The table below provides a quick reference of all components installed and their status:

Component	Package	Priority	Status
Apache Web Server	apache2	Must	✓ Installed
MariaDB Database	mariadb-server	Must	✓ Installed
PHP	php, php-mysql	Must	✓ Installed
phpMyAdmin	Manual install	Should	✓ Configured
MySQL Secure Install	mysql_secure_installation	Should	✓ Completed
File Permissions	chown / chmod	Should	✓ Applied
Dedicated DB User	webuser@localhost	Should	✓ Created
System Packages Updated	apt upgrade	Should	✓ Done
Self-Signed HTTPS	openssl + mod_ssl	Could	✓ Configured

10. Conclusion

The LAMP stack has been successfully installed and configured on the Ubuntu server. All Must and Should requirements from the User Story L4 work order have been fulfilled:

- Apache serves the test website and phpMyAdmin interface over the network.
- MariaDB is secured with mysql_secure_installation and a dedicated user with minimal rights.
- PHP processes dynamic content and interfaces with the database.
- phpMyAdmin was downloaded from the official website and configured manually.
- File permissions are set to prevent unauthorized access.
- All packages and the OS are up to date.
- HTTPS is configured with a self-signed certificate and HTTP redirects to HTTPS.

The server is accessible from the local network via both HTTP (redirected to HTTPS) and directly via HTTPS.